

# GraspSort

Vision-Guided Pick-and-Place Sorting with YOLO, RGB-D, MoveIt 2 and ROS 2

**One-line pitch:** a simulated robot arm looks at a cluttered table with an RGB-D camera, detects objects with YOLO, estimates each object's 3D pose, and picks and places every object into the bin for its class, using collision-aware motion planning in MoveIt 2.

**Purpose of this document:** the architecture reference for a second portfolio project, built with the same Claude Code workflow as SemNav. It deliberately covers a skill area SemNav did not: manipulation (kinematics, motion planning, grasping, planning scenes).

## 1. Why this project

- **New skill area:** SemNav proved perception, fusion and mobile navigation. GraspSort adds manipulation: inverse kinematics, Cartesian paths, collision objects, attach/detach, grasp candidates.
- **Reuse where it is cheap:** the YOLO ONNX Runtime node, the CycloneDDS + sysctl setup, the CLAUDE.md protocol and the ground-truth evaluation style carry over almost unchanged.
- **Measurable:** pick success rate, 3D pose error vs ground truth, planning time and cycle time on randomised layouts, so the README has real numbers.
- **Strong demo:** an arm sorting objects into bins reads instantly in a 30-second video.
- **Scope control:** fixed overhead camera (no hand-eye loop), top-down grasps, rigid simple objects, simulation only. No learned grasping, no dexterous hands.

## 2. Lessons from SemNav built into this plan

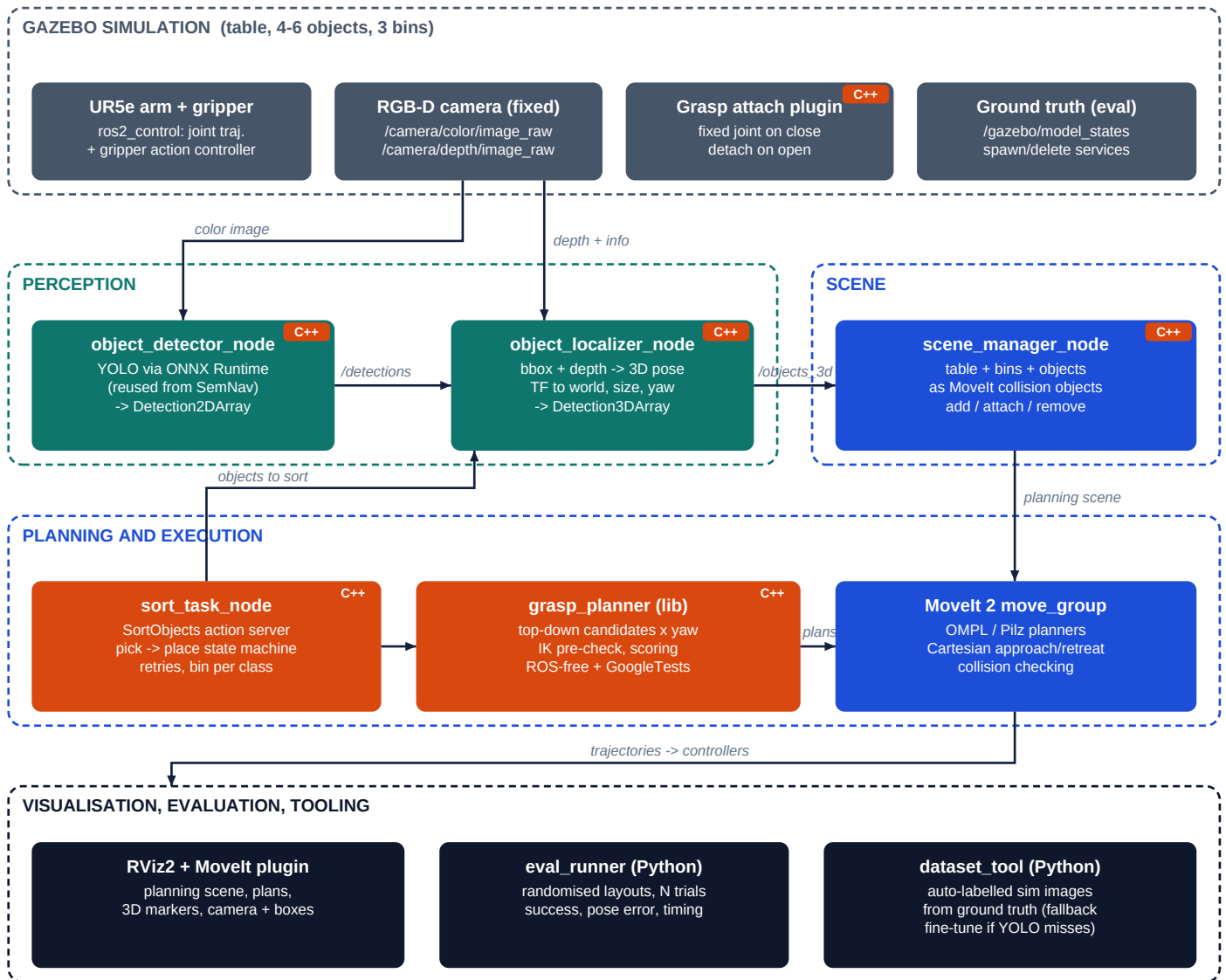
| SemNav lesson                             | How GraspSort applies it                                                                                                                          |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Validate the riskiest assumption on day 1 | Phase 0 checks: arm + gripper packages run in Gazebo, the grasp-attach mechanism holds an object, YOLO detects the chosen objects in sim images.  |
| YOLO missed the sim bottle                | Choose objects by measured detection rate, not by name. Fallback is an auto-labelled synthetic dataset and a short fine-tune.                     |
| Transport and TF issues cost hours        | Start on CycloneDDS with raised UDP buffers from the first launch. Keep the tf2 0.25.24 underlay available.                                       |
| Static map masked the semantic layer      | Objects never live in static scene files: the planning scene is built from perception only (table and bins are the only fixed collision objects). |
| Effect not shown by the A/B test          | Define the evaluation scenario before building, and make sure it can show the effect (randomised layouts, clutter levels).                        |
| Debugging rabbit holes                    | Every investigation gets a time box and a written fallback in DECISIONS.md.                                                                       |
| Manual demo recording failed              | A scripted demo (demo_sort.sh) and a demo RViz config are part of the plan, not an afterthought.                                                  |

## 3. Technology choices

| Item      | Choice                                                                                      | Notes / decision to confirm in Phase 0                                                                                |
|-----------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| OS / ROS  | Ubuntu 22.04 + ROS 2 Humble                                                                 | Same machine and setup as SemNav.                                                                                     |
| Simulator | Gazebo Classic 11 (default) or Gazebo Fortress                                              | D-01: pick whichever has a working UR5e + gripper + ros2_control simulation on this machine. Do not mix.              |
| Arm       | UR5e (Universal Robots ROS 2 packages)                                                      | Well documented MoveIt config. Franka Panda is the fallback if the UR sim does not run.                               |
| Gripper   | Parallel-jaw gripper (e.g. Robotiq 2F-85 description) driven by a gripper action controller | D-02: confirm description + controller availability; a simple custom 2-finger gripper URDF is an acceptable fallback. |

| Item                 | Choice                                                                                        | Notes / decision to confirm in Phase 0                                                                                                                   |
|----------------------|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Grasp holding</b> | Attach-on-close plugin (fixed joint between gripper and object)                               | Physics-only grasping in Gazebo is unreliable. Use a community link-attacher port or write a small C++ Gazebo plugin (a good C++ deliverable by itself). |
| <b>Planning</b>      | Movelt 2: move_group, MoveGroupInterface (C++), OMPL + Pilz LIN for straight approach/retreat | Movelt Task Constructor is a stretch goal, not the baseline.                                                                                             |
| <b>Perception</b>    | RGB-D camera, YOLOv8n via ONNX Runtime C++, OpenCV                                            | Reuse the SemNav detector node and its tests.                                                                                                            |
| <b>Middleware</b>    | CycloneDDS, raised UDP buffers                                                                | From SemNav D-19/D-20.                                                                                                                                   |
| <b>Build / test</b>  | colcon, ament_cmake, GoogleTest, GitHub Actions                                               | Pure logic (projection, grasp candidates, bin assignment) stays ROS-free and unit-tested.                                                                |

## 4. System architecture



**Reading the diagram:** orange "C++" tags mark code you write. Blue is MoveIt and the planning scene. Perception produces 3D object poses; the scene manager turns them into collision objects; the sort task node asks the grasp planner for candidates and MoveIt for collision-free trajectories, then executes them through ros2\_control.

### Data flow for one object

- The detector publishes 2D boxes for the latest colour frame (newest-frame buffer + worker thread, as in SemNav).
- The localizer takes the matching depth image, computes a robust depth inside the central part of each box, back-projects with the camera intrinsics, transforms to the world frame via TF and estimates footprint size and yaw from the depth mask.
- The scene manager adds or updates each object as a MoveIt collision object (cylinder or box) and keeps the table and bins as fixed objects.
- The sort task node picks the next object, gets ranked grasp candidates, and executes: pre-grasp -> straight-line approach -> close gripper -> attach -> lift -> move above the class bin -> release -> detach -> retreat.
- After each place, the object is removed from the planning scene and the camera view is re-checked before the next pick.

## 5. Packages and repository layout

```
graspsort_ws/ (repo root is the colcon workspace, as in SemNav D-11)
  CLAUDE.md  README.md  docs/{architecture.txt, CHECKLIST.md, DECISIONS.md, results.md, design-notes.md}
  scripts/   setup_ort.sh  export_yolo.py  fetch_models.sh  run_eval.sh  demo_sort.sh  make_dataset.py
  src/
    graspsort_msgs/          SortObjects.action, ObjectPose.msg, ObjectPoseArray.msg
    graspsort_perception/    object_detector_node (reused), object_localizer_node, projection.hpp + tests
    graspsort_scene/         scene_manager_node
    graspsort_manipulation/   sort_task_node, grasp_planner.hpp (ROS-free) + tests, bin_assignment.hpp + tests
    graspsort_gazebo/         world (table, bins, objects), arm+gripper+camera bringup, attach plugin (C++)
    graspsort_bringup/        launch/, config/ (moveit, controllers, perception params), rviz/
```

## 6. Interfaces

| Interface                             | Type                                                   | Producer -> Consumer          | Notes                                                                              |
|---------------------------------------|--------------------------------------------------------|-------------------------------|------------------------------------------------------------------------------------|
| /camera/color/image_raw               | sensor_msgs/Image                                      | Gazebo -> detector            | SensorData QoS; 640x480, 10-15 Hz                                                  |
| /camera/depth/image_raw + camera_info | sensor_msgs/Image, CameraInfo                          | Gazebo -> localizer           | Depth in metres (32FC1), same optical frame as colour                              |
| /detections                           | vision_msgs/Detection2DArray                           | detector -> localizer         | Header stamp copied from the image                                                 |
| /objects_3d                           | graspsort_msgs/ObjectPoseArray                         | localizer -> scene, sort task | world frame; class, confidence, pose, size, shape                                  |
| /planning_scene                       | moveit_msgs/PlanningScene (via PlanningSceneInterface) | scene -> move_group           | Table + bins fixed; objects from perception only                                   |
| /sort_objects                         | graspsort_msgs/action/SortObjects                      | user/script -> sort task      | Goal: classes to sort; feedback: current object, stage; result: per-object outcome |
| /gripper_controller/gripper_cmd       | control_msgs/action/GripperCommand                     | sort task -> ros2_control     | Open/close with effort limit                                                       |
| /attach, /detach                      | std_srvs or custom srv                                 | sort task -> attach plugin    | Fixed joint gripper <-> object in Gazebo                                           |
| /gazebo/model_states                  | gazebo_msgs/ModelStates                                | Gazebo -> eval only           | Ground truth; never used by the robot                                              |

### Custom messages

```
# ObjectPose.msg
std_msgs/Header header          # frame_id = "world"
uint32 id                      # e.g. "cup", "bottle"
string class_name
float32 confidence
geometry_msgs/Pose pose         # centre of the object, yaw only (upright objects)
geometry_msgs/Vector3 size      # x, y footprint and height (m)
uint8 shape                    # 0 = cylinder, 1 = box

# SortObjects.action
string[] classes                # empty = all known classes
---
uint32 picked
uint32 failed
string[] failure_reasons
---
string current_object
string stage                    # detect / plan_grasp / approach / grasp / transport / place
```

### Frames

```
world -> base_link (arm) -> ... -> tool0 -> gripper_tcp
world -> camera_link -> camera_color_optical_frame (fixed, from URDF; calibration is a stretch goal)
world -> table, bin_<class> (published as static frames for clarity)
```

## 7. Node specifications

### 7.1 object\_detector\_node (C++, reused)

- Copy from SemNav with its tests: letterbox, decode [1,84,8400], NMS, worker thread, /metrics p50/p95.
- Change only the class\_filter default to the GraspSort classes chosen in Phase 0, and the image topic name.

### 7.2 object\_localizer\_node (C++)

| Aspect            | Design                                                                                                                                                                                                                        |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Depth sampling    | Take depth pixels in the central 50% of the box; reject zeros/NaN; use the 20th percentile (nearest surface) rather than the median, so table pixels around a thin object do not pull the estimate (lesson from SemNav D-22). |
| 3D point          | Back-project the box centre with fx, fy, cx, cy from CameraInfo at that depth; push in by half the estimated object depth to approximate the centre.                                                                          |
| Footprint and yaw | Segment pixels within +/- 2 cm of the object depth inside the box, project them to the table plane, cv::minAreaRect for size and yaw (yaw ignored for cylinders).                                                             |
| Height            | Table plane height is known (parameter or fitted once); object height = table height - top surface.                                                                                                                           |
| Stability         | Objects are static: average over N frames, publish when the standard deviation is under a threshold. Track ids by nearest neighbour in 3D.                                                                                    |
| Frames            | Read frame_id from the image header; TF at the image stamp; fail loudly if TF fails (SemNav D-05 rule).                                                                                                                       |
| Tests             | GoogleTests for back-projection, percentile sampling with table pixels present, minAreaRect yaw, and stability gating.                                                                                                        |

### 7.3 scene\_manager\_node (C++)

- Adds the table and bins once at start-up as fixed collision objects (from parameters matching the world file).
- Adds or updates each perceived object as a cylinder or box; removes objects that are no longer seen; never adds objects from the world file directly.
- Supports attach/detach of the held object to the gripper link so MoveIt plans transport moves with the object in hand.
- Exposes a small service to freeze the scene during a pick, so a mid-motion perception update cannot move the target.

### 7.4 grasp\_planner (ROS-free C++ library)

| Aspect     | Design                                                                                                                                                                             |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Candidates | Top-down grasps only. For boxes: yaw aligned with the short side, plus 180 deg. For cylinders: 8 yaws around the axis. Grasp height from object height (fingers at 60% of height). |
| Filtering  | Reject candidates whose gripper opening is too small for the object width; reject if fingers would hit neighbouring objects (2D footprint check).                                  |
| Ranking    | Prefer yaw closest to the current wrist yaw (shorter motions) and largest clearance to neighbours.                                                                                 |
| Tests      | Box and cylinder candidates, width rejection, neighbour collision rejection, ranking order.                                                                                        |

### 7.5 sort\_task\_node (C++, MoveGroupInterface)

| Stage               | Implementation                                                                                                                                 |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Order               | Pick the object closest to the arm base first that is reachable (IK check) and not blocked.                                                    |
| Pre-grasp           | Pose 10 cm above the grasp with gripper open; plan with OMPL; try the next candidate if planning fails.                                        |
| Approach / retreat  | Straight-line Cartesian moves (computeCartesianPath or Pilz LIN), requiring at least 95% of the path.                                          |
| Grasp               | Close the gripper with an effort limit, call attach (Gazebo) and attachObject (MoveIt).                                                        |
| Transport and place | Move above the bin of the object's class (bin_assignment.hpp), lower, open, detach, retreat.                                                   |
| Recovery            | Up to 2 retries per object with the next grasp candidate; on failure report the reason in the action result and continue with the next object. |
| Feedback            | Publish stage and object in action feedback; log per-stage timings for the evaluation.                                                         |

### 7.6 Grasp attach plugin (C++ Gazebo plugin, or community link attacher)

- Creates a fixed joint between the gripper finger link and the object model on /attach, removes it on /detach.

- Phase 0 decides between an existing port and a ~150-line custom plugin. A custom one is a nice C++ deliverable and avoids an unmaintained dependency.

## 8. Evaluation plan

| Metric                   | How measured                                                                        | Target (set after baseline)          |
|--------------------------|-------------------------------------------------------------------------------------|--------------------------------------|
| <b>Pick success rate</b> | Objects ending in the correct bin / objects on the table, over N randomised layouts | At least 90% on low clutter          |
| <b>3D pose error</b>     | Localizer position vs /gazebo/model_states, per class                               | Median under 1.5 cm                  |
| <b>Yaw error (boxes)</b> | Estimated vs true yaw                                                               | Median under 10 deg                  |
| <b>Planning time</b>     | Per stage from sort_task_node logs                                                  | p95 under 2 s per plan               |
| <b>Cycle time</b>        | Seconds per object, detect to retreat                                               | Report mean and p95                  |
| <b>Failure breakdown</b> | Counts per failure reason (no IK, plan failed, grasp slipped, misdetection)         | Used to choose the next improvement  |
| <b>Clutter sweep</b>     | Repeat with 3, 5 and 7 objects, minimum spacing 2 cm vs 8 cm                        | Shows where the grasp filter matters |

Scenario design comes first (SemNav lesson): the clutter sweep is what can show the value of neighbour-aware grasp filtering. The eval\_runner randomises object positions through the Gazebo spawn service with a fixed seed per run.

## 9. Risks and mitigations

| Risk                                                             | Mitigation                                                                                                                                                         |
|------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Objects slip out of the gripper in Gazebo                        | Attach-on-close plugin (section 7.6); tested in Phase 0 before anything else.                                                                                      |
| UR5e + gripper + ros2_control sim does not run on Humble Classic | Try Gazebo Fortress variant, then Franka Panda; decide in Phase 0 (D-01).                                                                                          |
| YOLO does not detect the sim objects                             | Pick objects by measured detection rate; fallback: make_dataset.py renders auto-labelled images from ground truth and fine-tunes YOLOv8n for 1-2 hours on CPU/GPU. |
| Depth noise on thin objects                                      | Percentile sampling, temporal averaging and stability gating (7.2).                                                                                                |
| Movelt planning failures near bins and table                     | Pilz LIN for approach/retreat, sensible pre-grasp heights, IK pre-check, retries with the next candidate.                                                          |
| Gazebo + MoveIt CPU load                                         | Camera at 10 Hz, headless Gazebo for evaluation, RViz only for demos (with software GL on this laptop).                                                            |
| Claude Code writes code you cannot explain                       | Read each file; ask for an interview-style explanation; write design-notes.md yourself.                                                                            |

## 10. Build plan (about 9 working days)

| Phase | Goal                                                                                                             | Done when                                                   |
|-------|------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| 0     | Validate: arm + gripper + controllers in sim; attach mechanism holds an object; YOLO on sim frames; RGB-D topics | All three checks pass or a fallback is decided (D-01..D-04) |
| 1     | World: table, 3 bins, 4-6 objects, fixed RGB-D camera; MoveIt config loads; arm moves to named poses             | Arm moves from RViz MotionPlanning panel and from a script  |
| 2     | Messages + ROS-free libraries (projection, grasp_planner, bin_assignment) with GoogleTests                       | All unit tests green                                        |
| 3     | Perception: reused detector + object_localizer_node                                                              | 3D pose error measured vs ground truth                      |
| 4     | Scene manager + grasp attach; scripted single pick and place of one known object                                 | One object lands in a bin, 5 times in a row                 |
| 5     | sort_task_node with action interface, retries and feedback                                                       | Full table sorted from one action call                      |
| 6     | Evaluation: randomised layouts, clutter sweep, results.md                                                        | Results table with real numbers                             |
| 7     | Demo RViz config, demo_sort.sh, README, design notes, CI, recorded video, v1.0 tag                               | Stranger can run the demo in 15 minutes                     |

**Cut order if behind:** clutter sweep, yaw estimation for boxes (use cylinders only), fine-tuning fallback. **Never cut:** the attach check in Phase 0, unit tests, the success-rate table, the demo video.

## 11. Claude Code workflow

### CLAUDE.md (start from the SemNav one, change only these parts)

```
# GraspSort rules
Source of truth: docs/architecture.txt. Implement it; do not redesign it.
Stack: ROS 2 Humble, Ubuntu 22.04, Gazebo (per D-01), MoveIt 2, C++17, ament_cmake.
Middleware: RMW_IMPLEMENTATION=rmw_cyclonedds_cpp in every launch file.
Pure logic in ROS-free headers with GoogleTests. All tunables are parameters.
Objects are never added to the planning scene from world files; only from perception.
Every investigation has a time box; record findings and fallbacks in docs/DECISIONS.md.
Never run sudo; give me the exact command. Single branch main; commit per green slice; I push.
End every task with: files changed | commands + results | tests | issues | deviations | what I verify.
```

### Prompt style that worked in SemNav

- One phase per session, /clear between phases, short prompts that name the architecture section to follow.
- Independent work in parallel subagents (for example messages, ROS-free libraries and launch files), integration by the main agent.
- Always end with an objective check: tests, topic rates, ground-truth error, or action results, never "looks right".
- Hard stops on debugging, with the fallback written down before starting.

## First prompt (Phase 0)

Phase 0 for GraspSort. Read CLAUDE.md and docs/architecture.txt. Do NOT write project code yet.

Use parallel subagents to check, each with evidence:

A) Which UR5e (or Panda) + gripper + ros2\_control simulation runs here on Humble (Classic or Fortress).

B) A grasp-attach mechanism that holds an object while the arm moves (existing port or minimal plugin).

C) YOLOv8n detection rate on sim images of candidate objects (cup, bottle, can, box, ball).

Record D-01..D-04 in docs/DECISIONS.md with options and a recommendation, then stop and ask me.

## 12. Interview angle

---

- **Story:** "After a navigation project, I wanted to learn manipulation, so I built a vision-guided sorting arm and measured its success rate under increasing clutter."
- **Be ready to explain:** why top-down grasps, how depth becomes a 3D pose, why percentile depth instead of median, how MoveIt uses collision objects, why attach in Gazebo, and what the failure breakdown told you.
- **Stretch goals that add depth:** hand-eye calibration with an ArUco marker, MoveIt Task Constructor, a wrist camera for a closer second look, or a pose-estimation upgrade for non-upright objects.